

Agenda

1. Doubly linked list Introduction
2. Delete a given node from DLL
3. Insert a new node before tail of DLL
4. LRU Cache
5. Palindrome List

How is the starting point of the cycle found once a cycle is detected in the linked list?

4 options

Active Duration (Most preferred: 30 seconds)

Appears for 30 Secs

A

fast or
Reset the slow pointer to head and move both pointers one step at a time. ✓

B

Continue moving fast pointer until it reaches the head.

C

Move slow pointer two steps at a time.

Rahul	
Aatukuri Sai Sandeep	Harshita
Abishek Ilango	Titendra
Akash Deep Verma	
Anil	
ANUBHAV RAMNANI	
B.Adhi Shakthi	
Debasish Brahma	
Gautam Vysyaraju	
Harshad Sanjay Marathe	
Harshdeep Srivastava	
krishnamurthy Donta	
M S Haseeb Khan	
Mahesh Baswaraj	
Mohan Kumar M	
Nishant Raj	
Pranjul Kesharwani	
Prasanna	
Priyank Varshney	
Sahil Urade	
Saurabh Dayal	
SHIVAM SHIV	
Uddepta Saikia	
Vikas Yadav	
Yoshita Rathore	

Problems with SLL ?



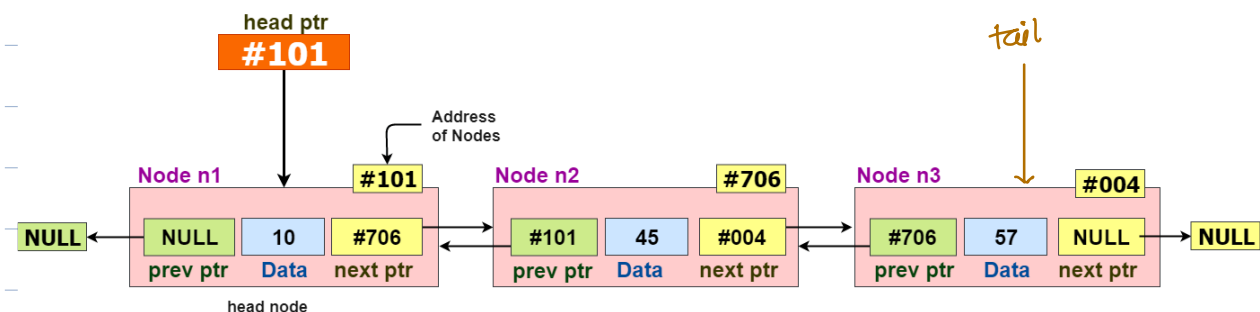
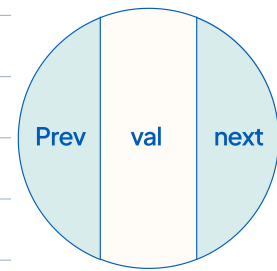
I don't know



Doubly LinkedList

Its SLL but with a prev pointer

```
class Node {  
    int val;  
    Node next;  
    Node prev;  
    public Node (int v) {  
        this.val = v;  
        this.next = null;  
        this.prev = null;  
    }  
}
```



Real life application

Spotify wants to enhance its user experience by allowing users to navigate through their music playlist seamlessly using "next" and "previous" song functionalities.

Problem

You are tasked to implement this feature using a doubly linked list where each node represents a song in the playlist. The system should support the following operations:

Add Song: Insert a new song into the playlist. If the playlist is currently empty, this song becomes the "Current song".

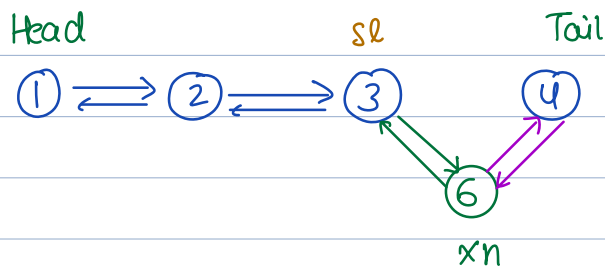
Play Next Song: Move to the next song in the playlist and display its details.

Play Previous Song: Move to the previous song in the playlist and display its details.

Current Song: Display the details of the current song being played.



Insertion in Doubly LinkedList just before tail



head, tail → You have head & tail nodes for DLL

```
void addBeforeTail (int x) {
```

```
    Node xn = new Node(x)
```

```
    secondLast = tail.prev
```

```
    secondLast.next = xn
```

```
    xn.prev = secondLast
```

TC: $O(1)$

SC: $O(1)$

```
    xn.next = tail
```

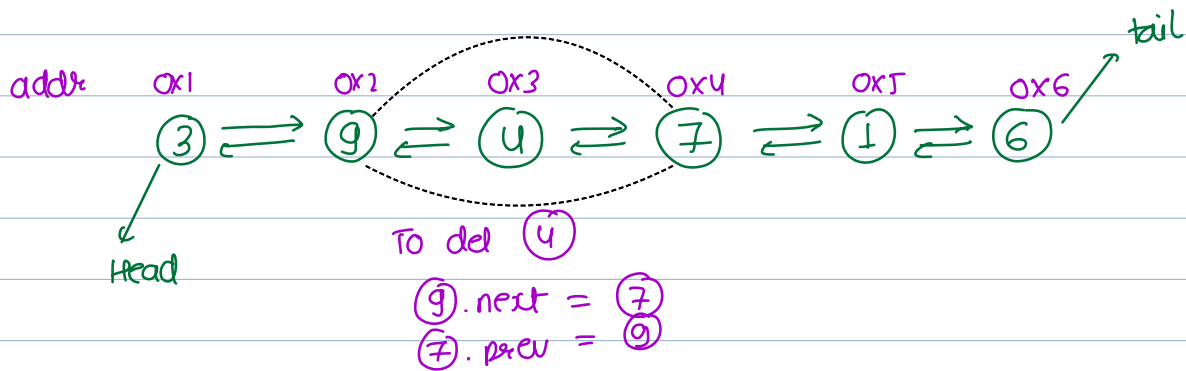
```
    tail.prev = xn
```

```
}
```



Delete a node in a DLL

head, tail



```
void deleteNode ( Node x ) {  
    |  
    x.prev.next = x.next  
    x.next.prev = x.prev  
    |  
    3
```

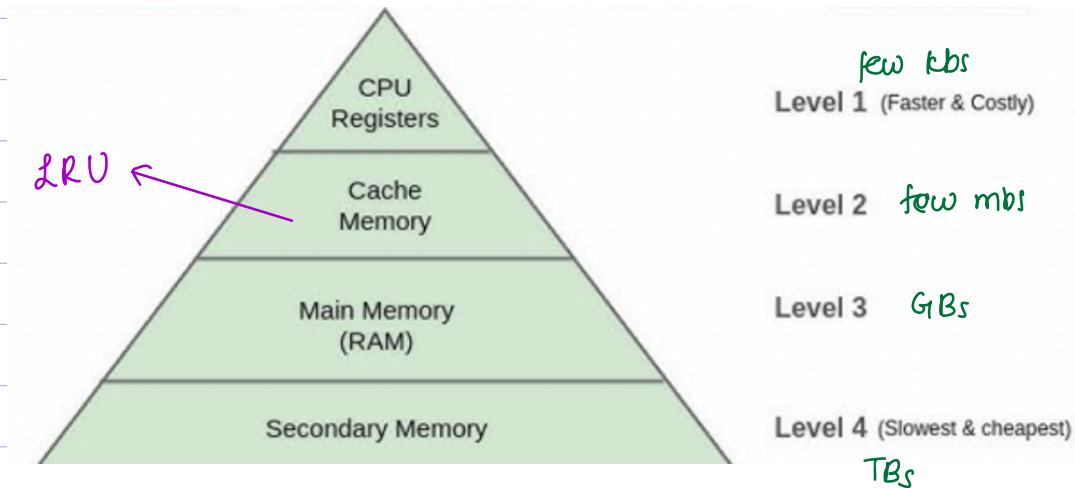
TC: $O(1)$

SC: $O(1)$

Note — we are not dealing with edge cases where head or tail may be deleted.



Memory Hierarchy



Least Recently Used — LRU cache operates on the principle that the data most recently used is likely to be accessed again in the near future.

LRU cache ensures that the most relevant data remains available in the cache.



L.R.U Cache *****

Design and implement a data structure for Least Recently Used (LRU) cache. It should support the following operations: get and set.

$O(1)$

get(key) - Get the value (will always be positive) of the key if the key exists in the cache, otherwise return -1.

$O(1)$

set(key, value) - Set or insert the value if the key is not already present. When the cache reaches its capacity, it should invalidate the least recently used item before inserting the new item.

The LRUCache will be initialised with an integer corresponding to its capacity. Capacity indicates the maximum number of unique keys it can hold at a time.

Definition of "least recently used" : An access to an item is defined as a get or a set operation of the item. "Least recently used" item is the one with the oldest access time.

Eg $\overset{k}{7} \quad \overset{v}{1}$ ✓

set (3, 2) ✓

set (9, 3) ✓

set (2, 4) ✓

set (6, 5) ✓

set (10, 6) ✓

set (14, 7) ✓

set (2, 8) ✓

get($\overset{k}{10}$) ✓ → 6

set (9, 19) ✓

set (15, 10) ✓

get($\overset{k}{2}$) ✓ → 8

capacity = 5

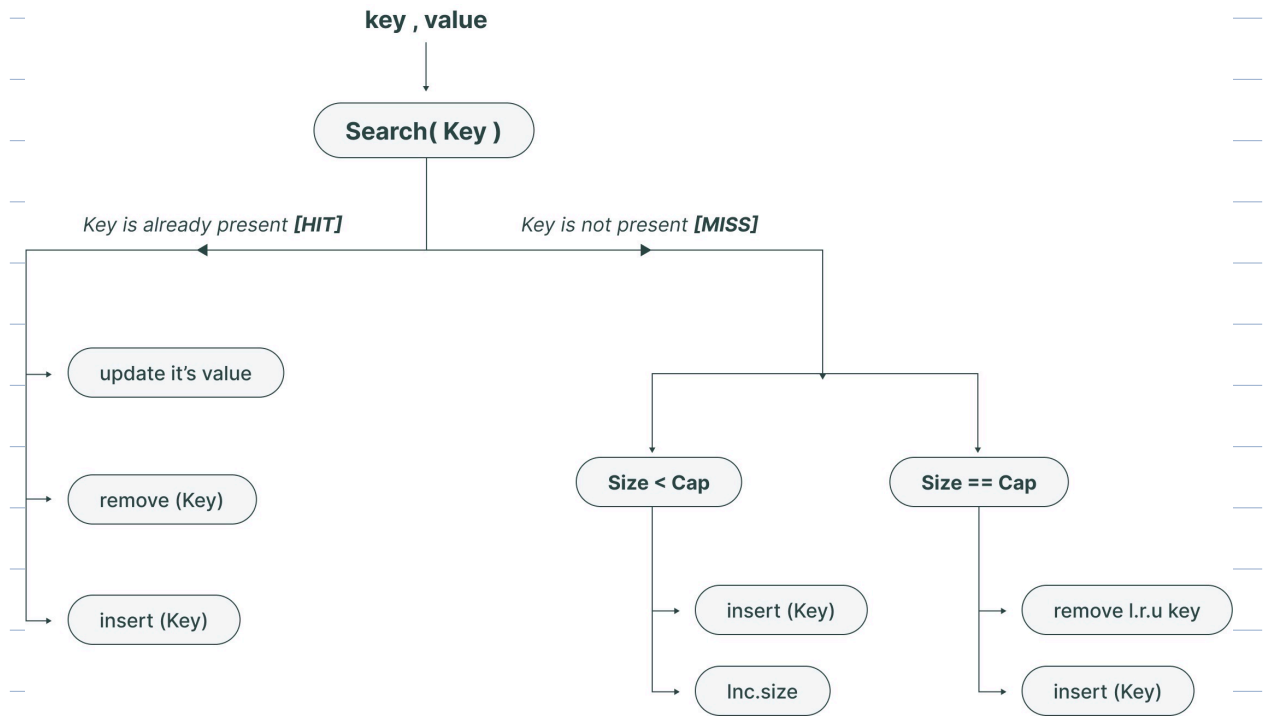
size = ~~5~~

(14, 7) (10, 6) ($\overset{19}{9, 2}$) (15, 10) ($\overset{8}{2, 4}$)

lru

mxu

	Queue				
	AL	HM	SLL	SL+HM	OLL+HM
search (k)	O(N)	O(1)	O(N)	O(1)	O(1)
add Last (k, v)	O(1)	X	O(1)	O(1)	O(1)
remove (k)	O(N)	O(1)	O(N)	O(N)	O(1)
Lru	O(1)	X	O(1)	O(1)	O(1)





**dry-run*

		✓	✓	✓	✓	✓	✓	✓	
k	[	10	15	19	20	15	18	23	20
		↓	↓	↓	↓	↓	↓	↓	
v		1	2	3	4	5	6	7	8

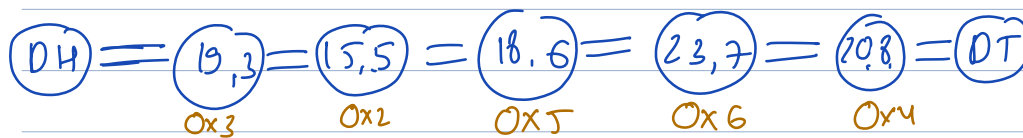
Capacity = 5

size = 0 ✓ 1 2 3 4 5

~~Head = null
tail = null~~

Hashmap

key	val {Node}
10	0x1
15	0x2
19	0x3
20	0x4
18	0x5
23	0x6



Break : 10:38



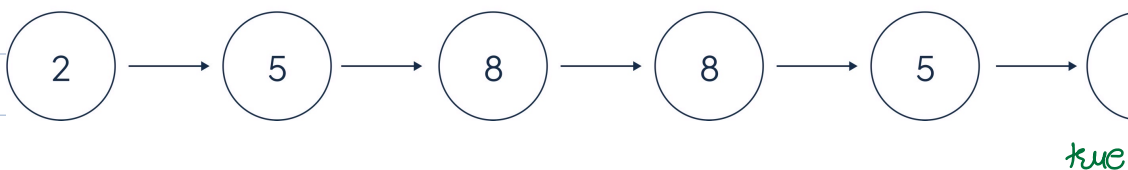
Pseudocode

Please refer to bookmark in the class video



SLL

- Check if a linked-list is a palindrome or not



Idea -1

Bruteforce {we arrays}

Make an array out of given linkedlist

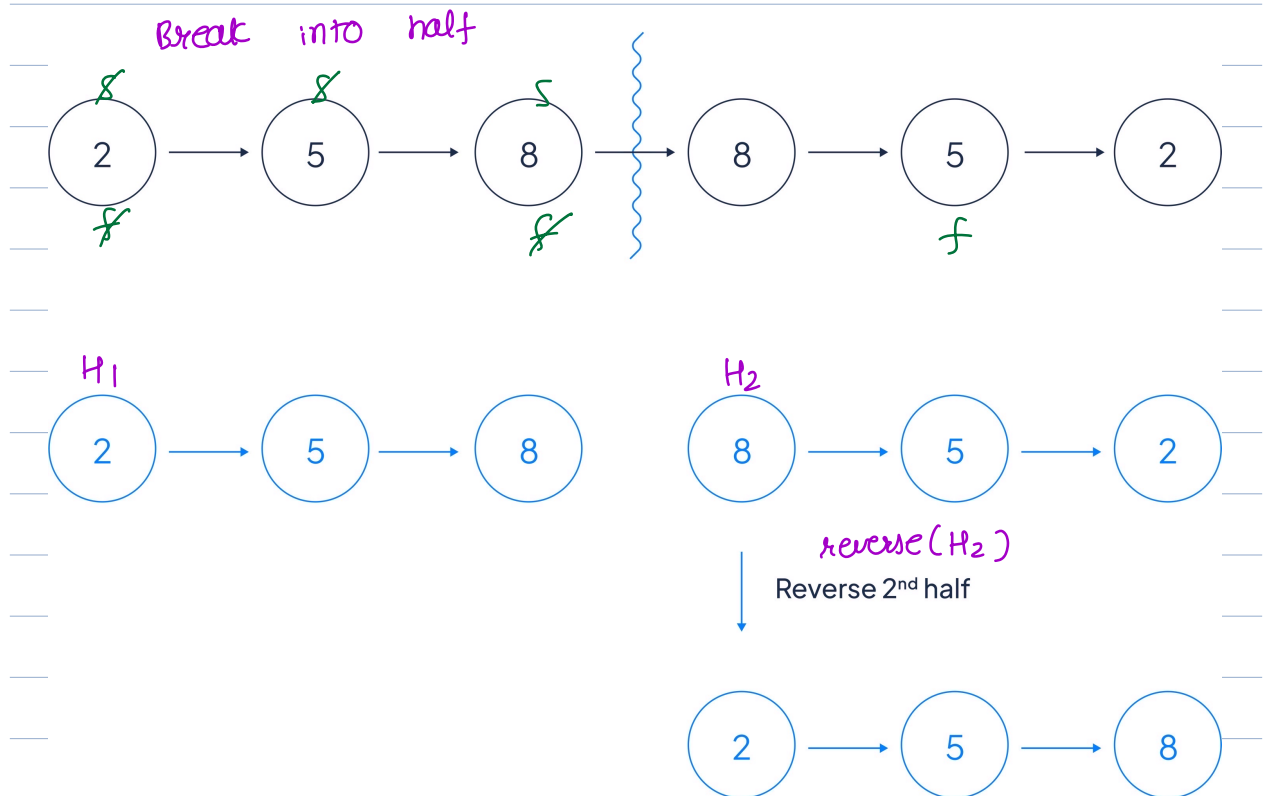
apply two pointers

Tc: $O(N)$

Sc: $O(N)$



Idea -2

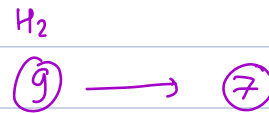
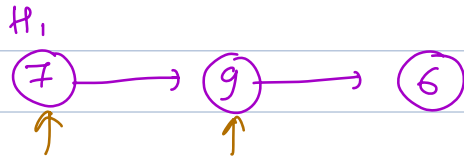
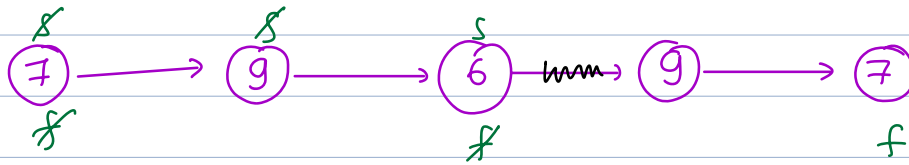


If first half and reversed second half are equal

→ return true;

otherwise

→ return false;



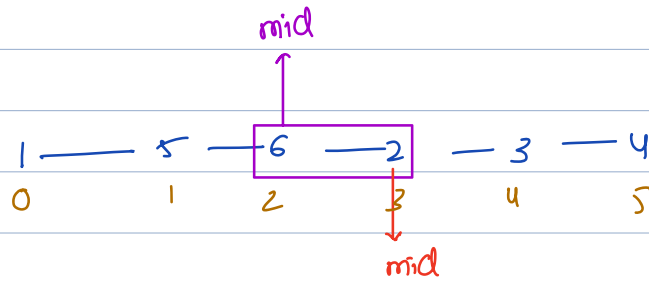
reverse (H_2)



TC: $O(N)$

SC: $O(1)$

Doubt session { Attendance is not affected during DS }

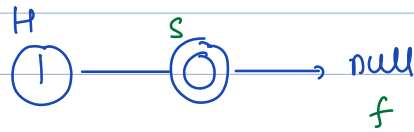
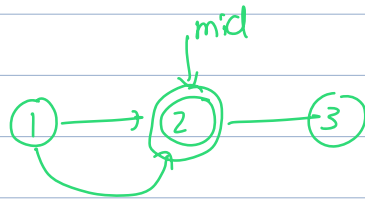


⑥

6/2

n/2

$$3/2 = 1$$



f or f.a.c.x.r.

mergeSortLL (H) {
Base X

